3D Object Recognition

3D Object Recognition

1. Content Description
2. Program Startup
3. Core Source Code Analysis

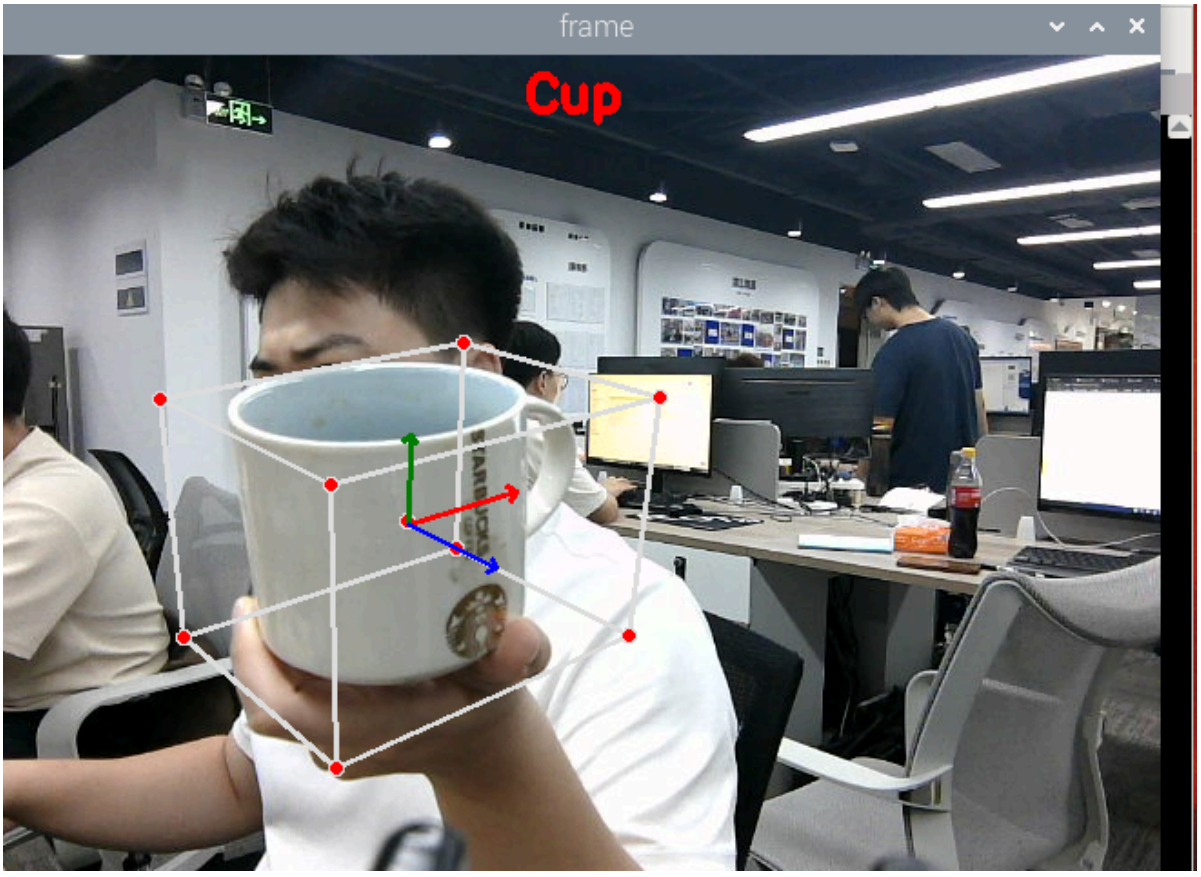
1. Content Description

This lesson implements the functions of acquiring the color image and using the mediapipe framework to recognize 3D objects (the case only shows four types: mug, shoe, chair, and camera).

2. Program Startup

```
ros2 launch yahboomcar_bringup mediapipe_demo.launch.py demo:=Objection
```

After the program runs, the default recognition target is the shoe (Shoe). You can press the F key to switch the recognized object. As shown in the figure below, a mug is recognized.



3. Core Source Code Analysis

Program code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/mediapipe/07_Objection.py
```

Import the required library files,

```
import mediapipe as mp
import cv2 as cv
import time
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import os
import numpy as np
```

Initialize the data and define the Publisher and Subscriber,

```
def __init__(self, staticMode=False, maxObjects=5, minDetectionCon=0.5, minTrackingCon=0.99):
    super().__init__('Objection')
    self.staticMode = staticMode
    self.maxObjects = maxObjects
    self.minDetectionCon = minDetectionCon
    self.minTrackingCon = minTrackingCon
    self.index = 0
    self.modelNames = ['Shoe', 'Chair', 'Cup', 'Camera']
    #Use the class in the mediapipe library to define a 3D detection object
    self.mpObjection = mp.solutions.Objection
    self.mpDraw = mp.solutions.drawing_utils
    self.mpObjection = self.mpObjection.Objection(
    self.staticMode, self.maxObjects, self.minDetectionCon, self.minTrackingCon, self.modelNames[self.index])
    self.bridge = CvBridge()
    #Define the Subscriber for the color image topic
    camera_type = os.getenv('CAMERA_TYPE', 'usb')
    topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type == 'nuwa' else '/usb_cam/image_raw'
    self.subscription = self.create_subscription(
    Image,
    topic_name,
    self.image_callback,
    10)
```

Color image callback function,

```
def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
    action = cv2.waitKey(1)
    #Press the F key to switch the recognized object
    if action == ord('f') or action == ord('F') : self.configUP()
    frame = self.findObjection(frame)
    cv.imshow('Objection', frame)
```

configUP function for switching the recognized object, selects self.modelNames by modifying the value of self.index. The options of self.modelNames are ['Shoe', 'Chair', 'Cup', 'Camera']

```
def configUP(self):
    self.index += 1
    if self.index>=4:self.index=0
        self.mpObjection = self.mpObjection.Objection(
        self.staticMode, self.maxObjects, self.minDetectionCon,
self.minTrackingCon,            self.modelNames[self.index])
```

findObjection object recognition function,

```
def findObjection(self, frame):
    cv.putText(frame, self.modelNames[self.index], (int(frame.shape[1] / 2) - 30,
30),
    cv.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255), 3)
    #Convert the color space of the input image from BGR to RGB for subsequent
image processing
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    #Call the process function in the mediapipe library for image processing;
during init, the self.pose object was created and initialized
    results = self.mpObjection.process(img_RGB)
    #Determine whether the target object is recognized
    if results.detected_objects:
        for id, detection in enumerate(results.detected_objects):
            #Draw a coordinate system on the recognized target object in the
image
            self.mpDraw.draw_landmarks(frame, detection.landmarks_2d,
self.mpObjection.BOX_CONNECTIONS)
            self.mpDraw.draw_axis(frame, detection.rotation,
detection.translation)
    return frame
```